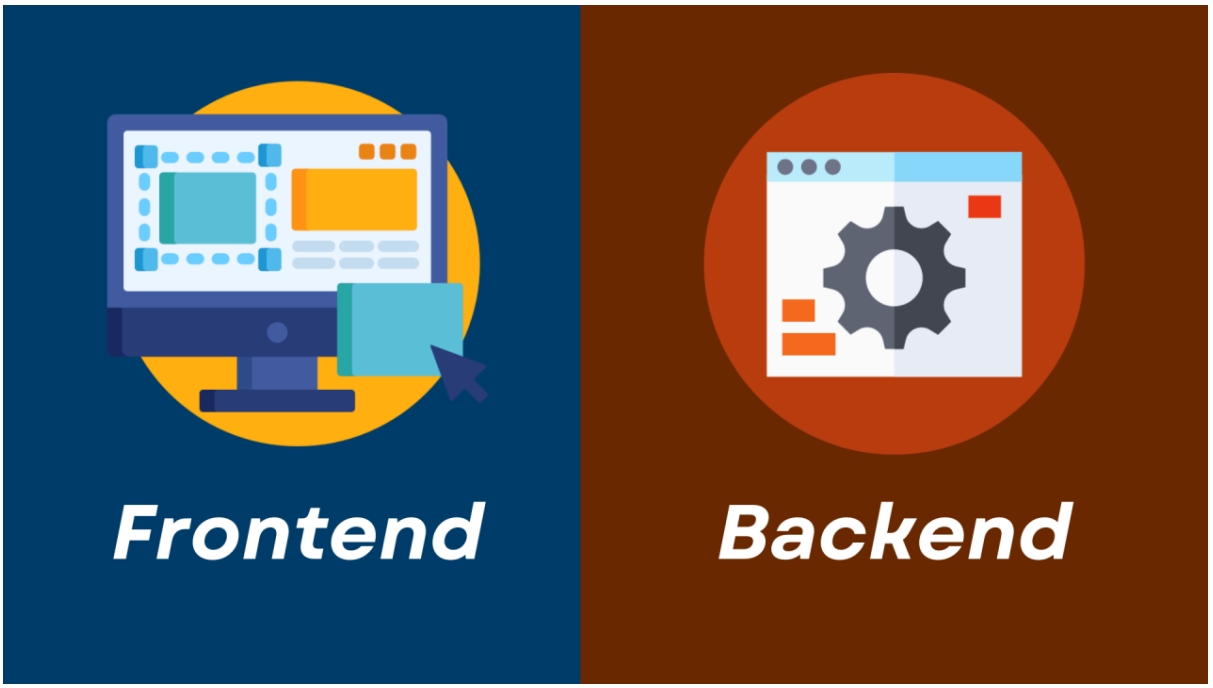
# OP1-B1-NODE-04




**Esteve Amargant Blanco**
**Maria Pilar Moreno**
**Frameworks de forntend i backends**

# Enunciat: Pràctica CRUD amb Express – Gestió de Productes

## Objectiu

Crear una **API REST** per gestionar un catàleg de productes amb **Express**, implementant totes les operacions **CRUD** (Create, Read, Update, Delete). Les dades es guardaran en memòria (en un array), sense base de dades.

## Planificació: 2 sessions

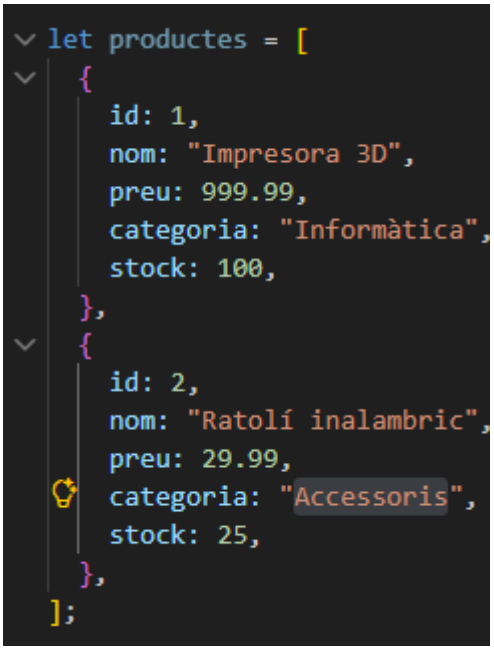
| Sessió | Què heu de tenir fet en acabar |
|---|---|
| **Sessió 1** | Projecte configurat, dades inicials, els dos GET, el POST (encara sense validacions) i la ruta 404. És el que hem vist a classe. |
| **Sessió 2** | PUT i DELETE (cal investigar-los, vegeu l'apartat 8), validacions, gestor d'errors, proves amb Postman i lliurament. |

## 1. Model Producte

Cada producte té les propietats següents:

```
{
    id: number,          // Identificador únic (el genera el servidor)
    nom: string,         // Nom del producte
    preu: number,        // Preu en euros
    categoria: string,   // Categoria del producte
    stock: number        // Quantitat disponible
}
```

L'`id` **no** l'envia mai el client: el genera el servidor en crear el producte, i no es pot modificar.

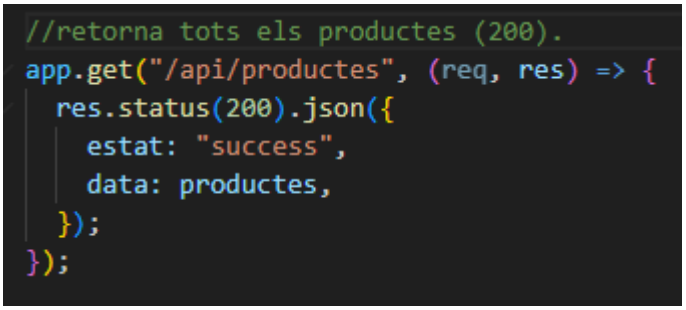


## 2. Endpoints a implementar

| Mètode | Ruta | Què fa | Codis de resposta |
|---|---|---|---|
| **GET** | `/api/productes` | Llista tots els productes | 200 |
| **GET** | `/api/productes/:id` | Retorna un producte | 200 · 400 id no vàlid · 404 |
| **POST** | `/api/productes` | Crea un producte | 201 · 400 dades invàlides |
| **PUT** | `/api/productes/:id` | Actualitza un producte | 200 · 400 · 404 |
| **DELETE** | `/api/productes/:id` | Elimina un producte | 200 · 400 id no vàlid · 404 |

### GET /api/productes

- Retorna tots els productes.
- Si no n'hi ha cap, retorna un array buit (amb codi 200, no és un error).



```
//retorna tots els productes (200).
app.get("/api/productes", (req, res) => {
  res.status(200).json({
    estat: "success",
    data: productes,
  });
});
```

**GET /api/productes/:id**

- Retorna el producte amb aquest id.
- Si l'id no és un número (per exemple `/api/productes/abc`), retorna **400**.
- Si l'id és vàlid però no existeix cap producte amb aquest id, retorna **404**.

```
//retorna tots els productes (200).
app.get("/api/productes", (req, res) => {
  res.status(200).json({
    estat: "success",
    data: productes,
  });
});

//retorna un producte concret.
app.get("/api/productes/:id", (req, res) => {
  const id = obtenirId(req);

  //400: l'id no és un número
  if (id === null) {
    return res.status(400).json({
      estat: "error",
      missatge: "L'id ha de ser un número enter",
    });
  }
  const producte = productes.find((p) => p.id === id);

  // 404: l'id és vàlid però no hi ha cap producte amb aquest id
  if (!producte) {
    return res.status(404).json({
      estat: "error",
      missatge: "Producte no trobat",
    });
  }

  res.status(200).json({
    estat: "success",
    data: producte,
  });
});
```

**POST /api/productes**

- El cos de la petició ha d'incloure: `nom`, `preu`, `categoria` i `stock`.
- Les dades s'han de validar (apartat 3). Si no són vàlides, retorna **400**.
- L'id s'ha de generar automàticament i ha de ser **únic**, fins i tot després d'esborrar productes (vegeu la pregunta de l'apartat 9).
- Retorna el producte creat, amb codi **201**.

```
//crea un producte nou o retorna 400 si les dades no són vàlides.
app.post("/api/productes", (req, res) => {
  const errors = validarProducte(req.body);
  if (errors.length > 0) {
    return res.status(400).json({
      estat: "error",
      missatge: "Dades invàlides",
      detalls: errors,
    });
  }

  const { nom, preu, categoria, stock } = req.body;

  const nouProducte = {
    id: seguentId++,
    nom: nom.trim(),
    preu,
    categoria: categoria.trim(),
    stock,
  };

  productes.push(nouProducte);

  res.status(201).json({
    estat: "success",
    missatge: "Producte creat correctament",
    data: nouProducte,
  });
});
```

**PUT /api/productes/:id**

- El cos de la petició ha d'incloure **tots** els camps (`nom`, `preu`, `categoria`, `stock`), amb les mateixes validacions que el POST.
- L'`id` no es pot modificar: si el cos inclou un `id`, s'ignora.
- Si el producte no existeix, retorna **404**.
- Retorna el producte actualitzat, amb codi **200**.

```
//actualitza un producte sencer, o retorna 400 / 404.
app.put("/api/productes/:id", (req, res) => {
  const id = obtenirId(req);

  if (id === null) {
    return res.status(400).json({
      estat: "error",
      missatge: "L'id ha de ser un número enter",
    });
  }

  const index = productes.findIndex((p) => p.id === id

  if (index === -1) {
    return res.status(404).json({
      estat: "error",
      missatge: "Producte no trobat",
    });
  }

  const errors = validarProducte(req.body);
  if (errors.length > 0) {
    return res.status(400).json({
      estat: "error",
      missatge: "Dades invàlides",
      detalls: errors,
    });
  }

  const { nom, preu, categoria, stock } = req.body;

  const producteActualitzat = {
    id,
    nom: nom.trim(),
    preu,
    categoria: categoria.trim(),
    stock,
  };

  productes[index] = producteActualitzat;

  res.status(200).json({
    estat: "success",
    missatge: "Producte actualitzat correctament",
    data: producteActualitzat,
  });
});
```

**DELETE /api/productes/:id**

- Elimina el producte.
- Si el producte no existeix, retorna **404**.
- Retorna un missatge de confirmació i el producte eliminat, amb codi **200**.

```
//elimina un producte, o retorna 400 / 404.
app.delete("/api/productes/:id", (req, res) => {
  const id = obtenirId(req);

  if (id === null) {
    return res.status(400).json({
      estat: "error",
      missatge: "L'id ha de ser un número enter",
    });
  }

  const index = productes.findIndex((p) => p.id === id);

  if (index === -1) {
    return res.status(404).json({
      estat: "error",
      missatge: "Producte no trobat",
    });
  }

  const producteEliminat = productes.splice(index, 1)[0];

  res.status(200).json({
    estat: "success",
    missatge: "Producte eliminat correctament",
    data: producteEliminat,
  });
});
```

## 3. Validacions requerides

S'apliquen al POST i al PUT. Cal comprovar tots els camps i retornar tots els errors alhora:

| Camp | Tipus | Condició |
|---|---|---|
| `nom` | `string` | No pot estar buit. Un text amb només espais (" ") també compta com a buit. |
| `preu` | `number` | Ha de ser un número positiu (més gran que 0). |
| `categoria` | `string` | No pot estar buida (tampoc només amb espais). |
| `stock` | `number` | Ha de ser un número enter positiu o zero. |

⚠️ **Compte amb els tipus:** `"preu": 29.99` és vàlid, però `"preu": "29.99"` (entre cometes, és un text) **no** ho és.

```
function validarProducte(body) {
  const errors = [];

  if (typeof body !== "object" || body === null || Array.isArray(body)) {
    errors.push("El cos de la petició ha de ser un objecte JSON");
    return errors;
  }

  const { nom, preu, categoria, stock } = body;

  if (typeof nom !== "string" || nom.trim() === "") {
    errors.push("El nom no pot estar buit");
  }

  if (typeof preu !== "number" || !Number.isFinite(preu) || preu <= 0) {
    errors.push("El preu ha de ser un número positiu");
  }

  if (typeof categoria !== "string" || categoria.trim() === "") {
    errors.push("La categoria no pot estar buida");
  }

  if (!Number.isInteger(stock) || stock < 0) {
    errors.push("L'stock ha de ser un número enter positiu o zero");
  }

  return errors;
}
```

## 4. Estructura de les respostes

Totes les respostes de l'API han de seguir aquest format (el mateix que hem fet servir a classe):

**Èxit**

```
if (index === -1) {
  return res.status(404).json({
    estat: "error",
    missatge: "Producte no trobat",
  });
}
```

**Error**

```
  res.status(201).json({
    estat: "success",
    missatge: "Producte creat correctament",
    data: nouProducte,
  });
});
```

**Error de validació (400)**

```
    return res.status(400).json({
      estat: "error",
      missatge: "Dades invàlides",
      detalls: errors,
    });
  }
```

## 5. Codis HTTP a utilitzar

| Codi | Quan |
|---|---|
| **200** | Operació correcta (GET, PUT, DELETE). |
| **201** | Recurs creat correctament (POST). |
| **400** | Error en les dades enviades: validació fallida, id no numèric o JSON mal format. |
| **404** | Recurs no trobat (producte inexistent o ruta inexistent). |
| **500** | Error intern del servidor (qualsevol error inesperat, recollit pel gestor d'errors). |

Tots implementats. El 404 i el 500 estan mes documentats al seguent punt de gestio d'errors

## 6. Gestió d'errors

### Ruta 404

Qualsevol ruta que no existeixi ha de respondre en JSON, amb codi 404 i el camí que s'ha demanat:

```
{
    "estat": "error",
    "missatge": "Ruta no trobada",
    "ruta": "/api/clients"
}
```

```
// Aixo va despres de les rutes anteriors per que si una petició arriba fins aquí
// vol dir que cap ruta anterior l'ha atesa, i per tant 404.
app.use((req, res) => {
  res.status(404).json({
    estat: "error",
    missatge: "Ruta no trobada",
    ruta: req.originalUrl,
  });
});
```

### Gestor d'errors

Cal un middleware d'errors (`app.use((err, req, res, next) => { ... })`) que:

- Respongui **400** en JSON quan el client envia un JSON mal format.
- Respongui **500** en JSON davant de qualsevol altre error, **sense** enviar al client els detalls interns (rutes de fitxers, traça de l'error...).

```
app.use((err, req, res, next) => {
  // 400: JSON mal format
  if (err.type === "entity.parse.failed") {
    return res.status(400).json({
      estat: "error",
      missatge: "El JSON enviat no és vàlid",
    });
  }
  // 500: qualsevol altre error.
  console.error(err);
  res.status(500).json({
    estat: "error",
    missatge: "Error intern del servidor",
  });
});
```

## 7. Ordre del fitxer app.js

A Express l'ordre importa. El vostre app.js ha de seguir aquesta estructura:

```
1. Imports i configuració          require, express(), PORT
2. Middleware                      app.use(express.json())
3. Dades inicials                  let productes = [...]
4.  Rutes                                  app.get / app.post / app.put /
app.delete
5. Ruta 404                        app.use((req, res) => ...)
6. Gestor d'errors                 app.use((err, req, res, next) => ...)
7. Engegar el servidor             app.listen(...)
```

**Dades inicials**

```
let productes = [
    {
        id: 1,
        nom: "Portàtil Gaming",
        preu: 999.99,
        categoria: "Informàtica",
        stock: 10
    },
    {
        id: 2,
        nom: "Ratolí sense fil",
        preu: 29.99,
        categoria: "Accessoris",
        stock: 25
    }
];
```

## 8.  Per investigar

El PUT i el DELETE no els hem fet a classe: us toca investigar-los. Aquestes eines us seran útils:

| Eina | Per a què us pot servir |
|---|---|
| `app.put()` · `app.delete()` | Definir rutes per a aquests mètodes (funcionen com app.get i app.post). |
| `array.findIndex()` | Trobar la posició d'un producte dins l'array. |
| `array.splice()` o `array.filter()` | Eliminar un element de l'array. |
| `typeof` | Comprovar si un valor és un number o un string. |
| `Number.isInteger()` | Comprovar si un número és enter (per a l'stock). |
| `Number.isNaN()` | Detectar un id que no és un número (parseInt("abc") és NaN). |
| `string.trim()` | Treure els espais del principi i del final d'un text. |

**Documentació:** expressjs.com (Express 5) i developer.mozilla.org (JavaScript).

## 9. Checklist de proves amb Postman

Feu totes aquestes proves, guardeu-les a la col·lecció de Postman i incloeu-ne la captura al PDF:

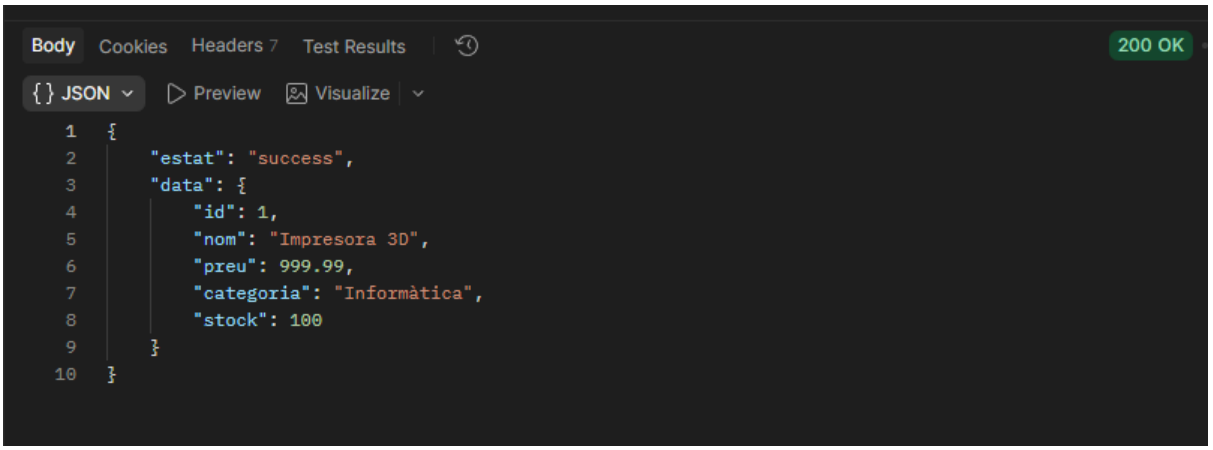
| # | Petició | Resultat esperat |
|---|---|---|
| 1 | `GET /api/productes` | 200 + llista de productes |
| 2 | `GET /api/productes/1` | 200 + el producte 1 |
| 3 | `GET /api/productes/999` | 404 |
| 4 | `GET /api/productes/abc` | 400 |
| 5 | `POST /api/productes` amb totes les dades correctes | 201 + producte amb id nou |
| 6 | `POST` amb el nom buit o només espais | 400 + detalls |
| 7 | `POST` amb el preu negatiu o entre cometes | 400 + detalls |
| 8 | `POST` amb un stock decimal (`2.5`) | 400 + detalls |
| 9 | `POST` amb diversos camps incorrectes alhora | 400 + tots els errors |
| 10 | `POST` amb un JSON mal format (sense la `}` final) | 400 en JSON |
| 11 | `PUT /api/productes/1` amb dades correctes | 200 + producte actualitzat |
| 12 | `PUT /api/productes/999` | 404 |
| 13 | `PUT /api/productes/1` amb dades incorrectes | 400 + detalls |
| 14 | `DELETE /api/productes/2` | 200 + confirmació |
| 15 | `GET /api/productes/2` (després d'esborrar-lo) | 404 |
| 16 | `DELETE /api/productes/999` | 404 |
| 17 | `GET /api/clients` (ruta inexistent) | 404 + ruta |

> 🤔 **Pregunta de reflexió (responeu-la al PDF)**
> Engegueu el servidor de nou, esborreu el producte 1 i després creeu dos productes nous. Quins ids han rebut? Es repeteix algun id? Expliqueu com genereu l'id i per què és (o no és) segur.

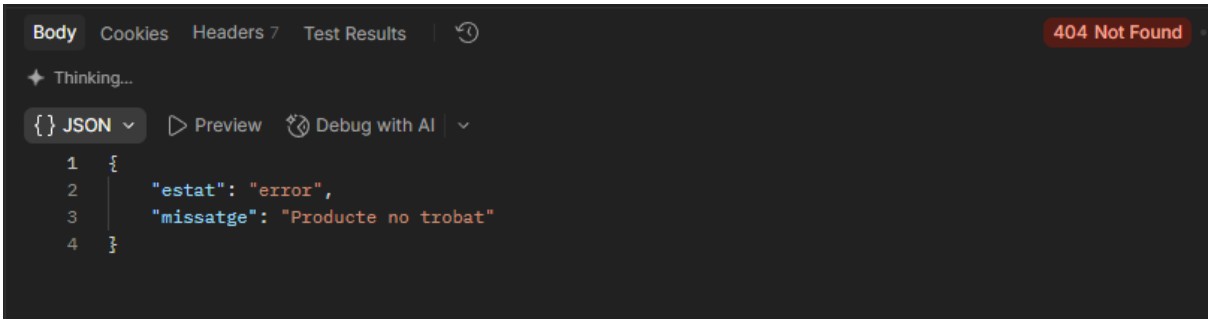
PETICIO 1



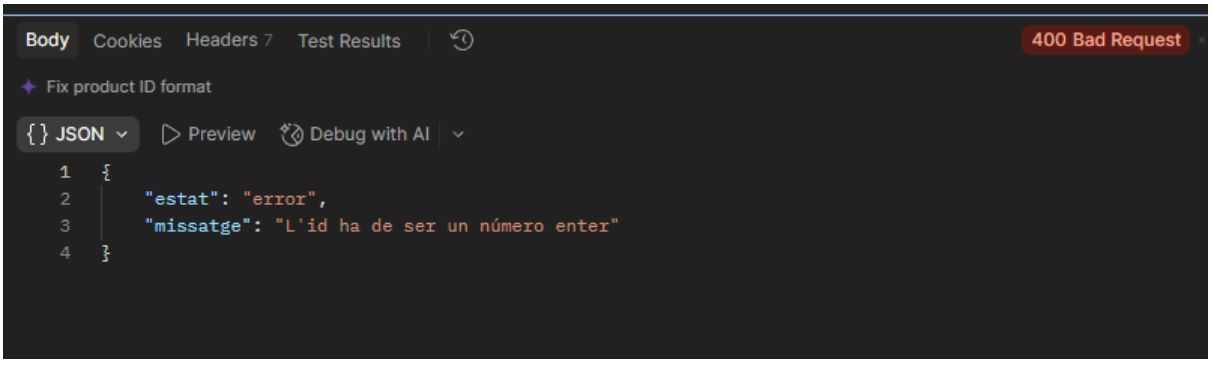

PETICIO 2




PETICIO 3




PETICIO 4



PETICIO 5



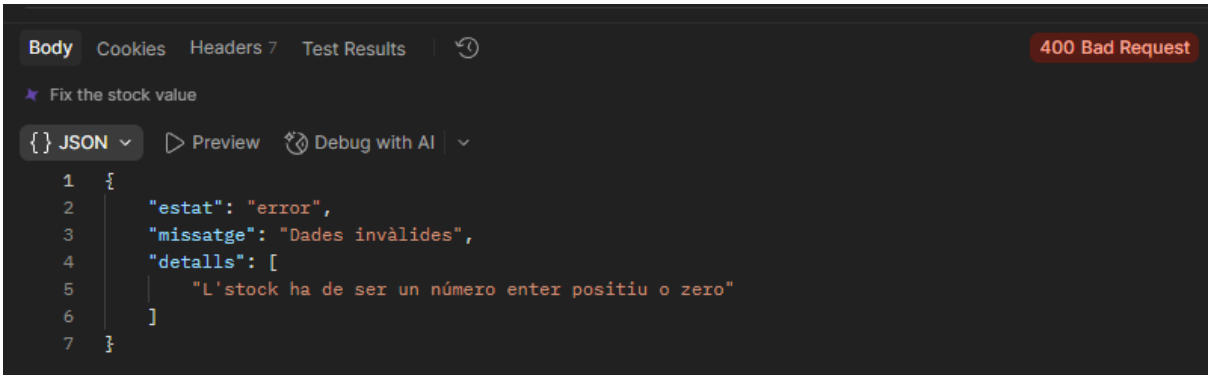

PETICIO 6



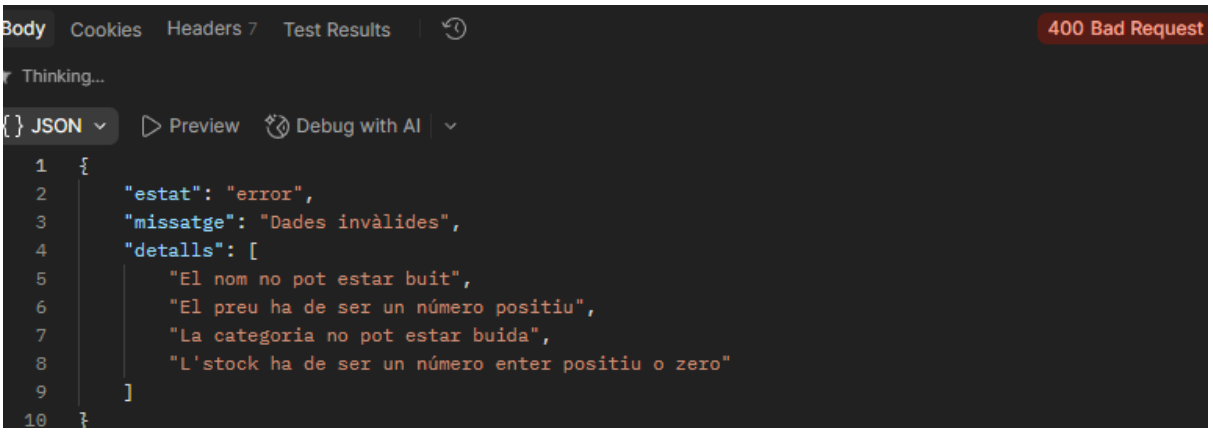

PETICIO 7



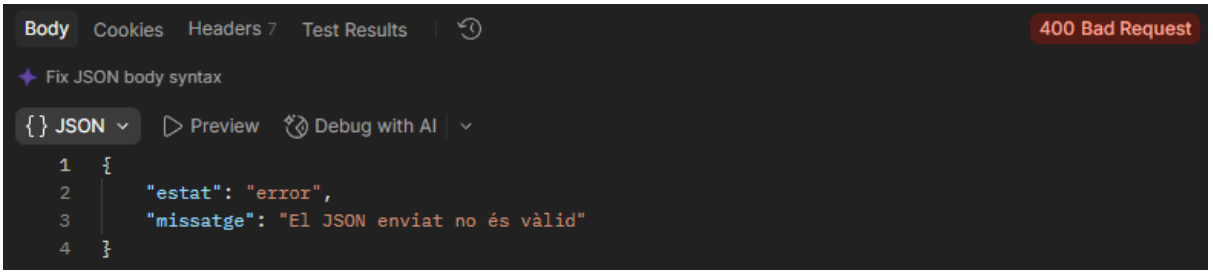

PETICIO 8



PETICIO 9



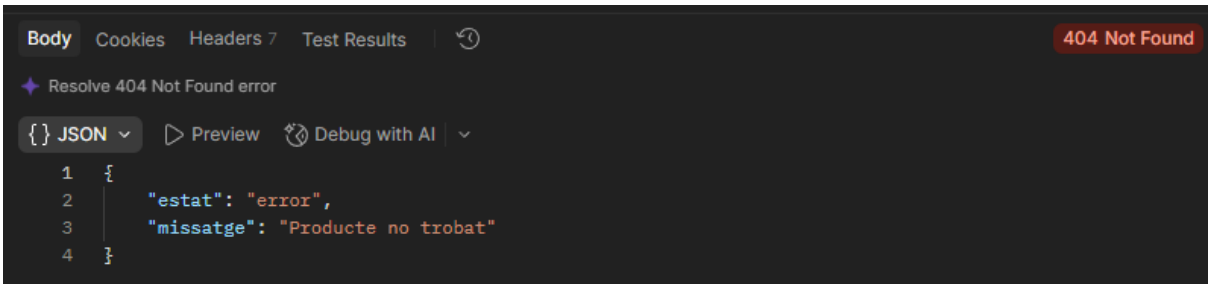

PETICIO 10




PETICIO 11



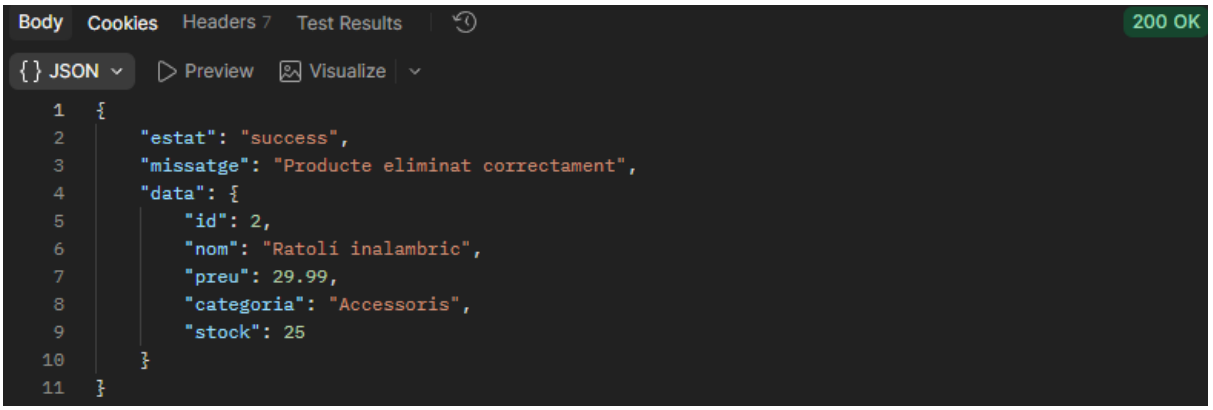

PETICIO 12



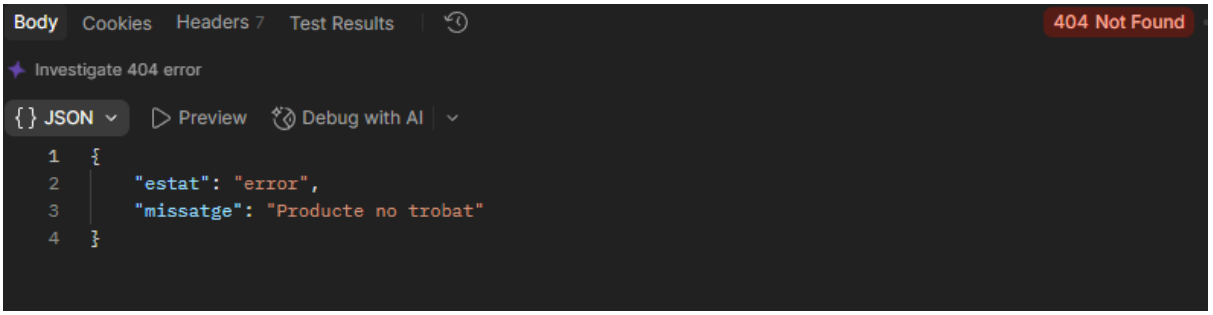

PETICIO 13




PETICIO 14



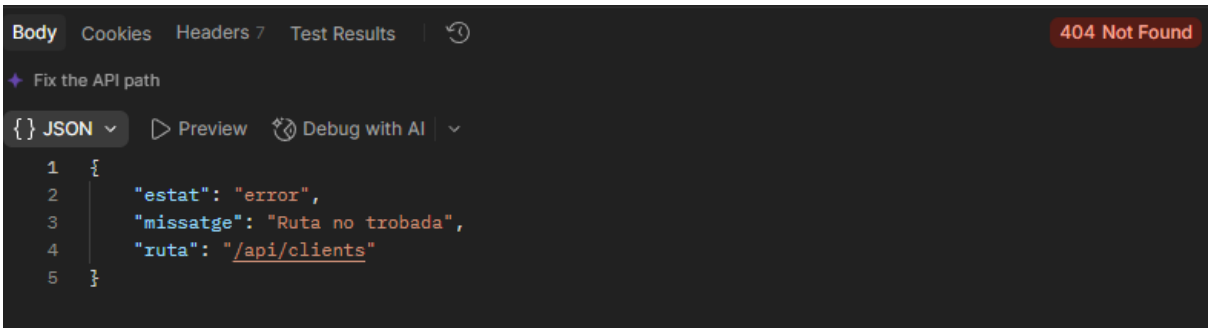

PETICIO 15




PETICIO 16



PETICIO 17




🤔 **Pregunta de reflexió (responeu-la al PDF)**

Cada cop que posem ne marxa el servidor tenim les dades carregades (Els dos productes) i no son dades persistents, es a dir si en una sessio els esborrem, no seguiran esborrats a la seguent. Cada cop que posem en marxa el servidor els tindrem alla, i els id seguiran a partir del 2, es a dir, el primer porducte qu eafegim al iniciar el servidor via post, tindra el id 3. Aixo sempre sera aixi fins que implementem una DB i guardem les dades en un altre lloc i persisteixxin entre sessions.

# Explicació del package.json: què fa cada script i cada dependència.

```
{
  "name": "servidor-Frameworks", → Nom del projecte
  "version": "1.0.0",
  "description": "API REST CRUD de productes amb Express",
  "main": "app.js", →  fitxer principal, pero on entra la applicacio
  "scripts": {
    "start": "node app.js", → Engega el servidor - npm run start
    "dev": "node --watch app.js" → Engega el servidor i actualitza els
canvis automaticamnet - npm run dev
  },
  "keywords": [],
  "author": "Esteve Amargant", → Jo
  "license": "ISC", → per defecte
  "type": "commonjs", → require per importar
  "dependencies": { → les dependencies del projecte, quines utilitzems
    "express": "^5.3.0"
  }
}
```